PFP X: Exact Host Checking of Model Verdicts on Small Finite Algebraic Challenges — Lattice Holonomy and Six Formal-Mathematics Pilots

Jeremy H. Carroll

2026-10-02 · Version 0.1.0 · Research methods paper · Author-review revision

Code baseline: `a623568eb0f853b968bc419bf06dfc6af256ab02`. Each model run discussed in Sections 3 and 4 is cited by its repository path and by the commit that recorded it; all of those commits precede this baseline.

Version 0.1.0 designates the initial public preprint candidate. Earlier 1.x and 2.x numbers in the change log identify internal drafts and corrections, not prior Zenodo publications. No DOI or external publication is claimed.

Abstract

Exact host checking separates a model's proposed verdict from a verified certificate. For finite-group connections on square lattices, this paper provides a complete Python flatness checker over a validated multiplication table. It checks ordered plaquette products with inverses on reversed edges; on a torus, two cycle holonomies distinguish gauge-trivial from other flat connections. Skipping four-identity-link plaquettes is an optimization whose saving depends on the gauge. On five retained C017 cases, fresh host replay reproduces the expected verdicts and the historical harness decisions on all 15 retained solution records: 13 parsed certificates and two parser-error records, with 10 accepted and 5 rejected overall. Ten further historical responses retain only labels and reasons. The five runs accepted 0, 0, 3, 3 and 4 responses out of five under different models, prompts and budgets; they do not identify a causal prompt effect. Six formal-mathematics pilots report three models on ten cases per cell, with two C049 panels and two C050 runs. The author's case-by-case annotation identifies answer-bearing identifiers in 36 of 60 reconstructable panel cases. In two challenges a constant refusal passes all ten under the original checkers. These counts therefore do not measure unaided performance. The package supplies retained reports, offline table recounts, negative controls, and one narrowly scoped Lean identity-link lemma. Unsupported benchmark and effect claims from the initial internal draft are withdrawn. No model run is repeated for this revision.

1. Problem and design

The Proposal Fidelity Protocol (PFP), developed in this collection under the legacy name Symbol Game, separates a candidate answer from the host check that decides whether the answer is accepted. This paper applies that separation to model answers about small finite algebraic objects. The model states a verdict and a certificate. A deterministic host program recomputes the property from the problem data using only exact operations: table lookups and integer arithmetic. The decision depends on the certificate, not on the model's explanation.

Lattice holonomy is a useful test of this design. For a nonabelian group, the product around a plaquette depends on the order of multiplication and on the orientation, and an error in either gives a confident wrong answer. Checking is cheap: one pass over the plaquettes and, on a torus, two cycle products. The task therefore shows both whether a host catches order errors and what a harness must tell a model for its answer to be checkable at all.

The paper makes three contributions:

1. a complete flatness check for finite-group connections on square lattices, with tests (Section 2);
2. an exact replay of the five retained C017 lattice cases and of every retained model solution record for them, with per-arm counts (Section 3);
3. a table of the retained formal-mathematics pilot runs for challenges C045 to C050, with what the paired checkers accepted, what the prompts disclosed and the limits the harness set (Section 4).

No new model runs were made for this revision. The earlier internal revision numbered 2.0.1 added a replay, with no model call, of answers that need no mathematics against the C045 to C050 checkers as they were when the runs were recorded (Section 4.2).

The initial internal draft of this paper (2026-09-28) also described a five-stage prompt filter whose stages were named after immune mechanisms, and it reported a 50 of 50 benchmark, a 40 per cent reduction in premature rollbacks, an 83.5 per cent reduction in prompt tokens and a 50 per cent accuracy gain. No run, runner or receipt for any of these is retained in the repository, so they are withdrawn; the change log lists every removed statement. In the initial internal draft's package the stages were simple functions. "MHC peptide decomposition" split an edge list into radius-one neighbourhoods. "Thymic selection" tested membership in a supplied list. "Kinetic proofreading" compared two supplied fractions with two thresholds. The "Treg scratchpad" check tested that four labelled lines appear in order. The "allosteric quorum" counted how many of five listed Unicode symbols occur in the prompt and compared the count with three. No effect of any of them on model answers was measured.

2. A complete flatness check

2.1 Definitions

A finite group G is given by named elements, a full multiplication table and an inverse map. The package checks the identity law and the inverse law for every element and associativity for every triple before it uses a table, and it refuses a table that fails. It builds S_3 from the permutations of $\{0,1,2\}$ and the quaternion group Q_8 from the unit quaternions $\pm 1, \pm i, \pm j, \pm k$ with integer coordinates.

An $L \times M$ lattice has vertices (r, c) with $0 \le r < L$ and $0 \le c < M$. Its positive edges are $(r, c) \to (r, c+1)$ and $(r, c) \to (r+1, c)$; on a torus the indices are taken modulo L and M. An open lattice requires $L, M \ge 2$ in this implementation. A torus requires $L, M \ge 3$: at size two, distinct parallel edges would have the same endpoint names and need a different representation. A connection assigns an element $U_{uv} \in G$ to each positive edge, and a reversed edge carries the inverse, $U_{vu} = U_{uv}^{-1}$. The product along a path $v_0 \to v_1 \to \cdots \to v_k$ is

$$\mathrm{hol}(v_0 \to \cdots \to v_k) = U_{v_0 v_1} U_{v_1 v_2} \cdots U_{v_{k-1} v_k}.$$

The plaquette $P_{r,c}$ is listed as $v_0 = (r, c)$, $v_1 = (r+1, c)$, $v_2 = (r+1, c+1)$, $v_3 = (r, c+1)$, and its product is

$$\mathrm{hol}(P_{r,c}) = U_{v_0 v_1} U_{v_1 v_2} U_{v_2 v_3} U_{v_3 v_0}.$$

The last two factors traverse positive edges backwards, so they are the inverses of the stored links. A connection is flat when every plaquette product is the identity e.

A gauge g assigns an element to each vertex and acts by $U'_{uv} = g_u U_{uv} g_v^{-1}$. The product around a closed path based at v_0 becomes $g_{v_0} \operatorname{hol} g_{v_0}^{-1}$. Flatness is therefore unchanged by a gauge transformation. On a torus, let A be the product around $(0,0) \to (0,1) \to \cdots \to (0,M-1) \to (0,0)$ and B the product around $(0,0) \to (1,0) \to \cdots \to (L-1,0) \to (0,0)$. A gauge transformation conjugates A and B by the same element.

We use four standard facts [1, 2, 3]. Reversing a closed path inverts its product. Starting the same closed path at v_1 instead of v_0 conjugates its product by $U_{v_0 v_1}^{-1}$. If a connection is flat, the product along a closed path depends only on the path's homotopy class, so on an open lattice, which is simply connected, a gauge built along a spanning tree makes every link the identity. On the torus, whose fundamental group $\mathbb{Z}^2$ is abelian, a flat connection has commuting A and B, and it is gauge-trivial exactly when $A = B = e$.

2.2 The check

The checker in `code/holonomy.py` returns one of three verdicts.

1. It reads the group table and refuses it unless it is a group. It reads the links and refuses them unless every edge is listed in both directions with $U_{vu} = U_{uv}^{-1}$ and no other edge is listed. Every value must be a declared group element; duplicate parsed edges are refused even if alternate strings such as `0,0->0,1` and `00,0->0,1` carry equal values.
2. It computes every plaquette product in the order of Section 2.1. If any is not e, it returns `NOT_FLAT` with the first such plaquette in row-major order and its product.
3. On an open lattice it returns `FLAT_TRIVIAL` with the gauge $g_{(0,0)} = e$, $g_v = g_u U_{uv}$ along a breadth-first spanning tree, after verifying that this gauge makes every link the identity.
4. On a torus it computes A and B and stops with an error if they do not commute, which cannot happen for a flat connection. If $A = B = e$ it returns `FLAT_TRIVIAL` with the verified tree gauge; otherwise it returns `FLAT_NONTRIVIAL` with A and B.

`FLAT_TRIVIAL` is the verdict that challenge C017 calls `PURE_GAUGE`.

A separate function checks a stated verdict and its certificate. For `NOT_FLAT` the four listed vertices must bound a plaquette, in either orientation and from any starting vertex, and the stated product must equal the product in the listed order and differ from e. For `FLAT_NONTRIVIAL` every plaquette must be flat and at least one stated closed loop must have the stated, non-identity product. In a flat connection every contractible loop has product e, so such a loop is non-contractible. For `FLAT_TRIVIAL` the stated gauge must make every link the identity. Any other answer, including an answer that could not be parsed, is rejected with a reason.

A gauge certificate must cover each vertex once after parsing; an alias may not silently replace a conflicting value. These checks concern parsed mappings; raw JSON duplicate-key handling is the responsibility of the caller's JSON decoder.

Correctness and cost. A nonidentity plaquette directly witnesses non-flatness. If all plaquettes are identity, deleting an immediate backtrack or replacing a path segment by the other sides of a plaquette preserves its product. On the open rectangle these moves contract every closed path, so root-to-vertex products are path-independent and the tree gauge trivializes all edges. On the torus, closed paths reduce to the two generators with the commutator relation: their products commute, and all closed products are identity exactly when both generators are. This proves the three-way decision for the declared rectangular cell complexes using the standard facts of Section 2.1; it is not a formal proof of the Python implementation. Table validation costs $O(|G|^3)$ operations and $O(|G|^2)$

table storage. Once the group is validated, the lattice decision costs $O(LM)$ table operations and storage. A certificate check costs $O(LM + k)$, where k is the total number of submitted loop edges; malformed inputs receive a typed refusal rather than a flatness verdict.

2.3 Remark: skipping plaquettes with four identity links

If the four links of a plaquette are the identity, its product is the identity. This uses only the identity law and $e^{-1} = e$; the package records the one-line proof in Lean (`identity_links_give_identity_product` in `lean4/LatticeDecimation.lean`). The initial internal draft called this observation a theorem. It is a direct consequence of the group axioms.

The checker may skip such plaquettes. It then performs fewer multiplications and reaches the same result, which the tests confirm on every connection they build. The saving depends on the gauge. If $U = e$ and g is a gauge, then $U'_{uv} = g_u g_v^{-1}$ is the identity only when $g_u = g_v$. A gauge that differs on neighbouring vertices therefore leaves no plaquette with four identity links, while the connection stays flat and gauge-trivial. One retained C017 case, `base_02`, has this form: 28 of its 31 links are not the identity, none of its 12 plaquettes can be skipped, and it is gauge-trivial. Skipping is an optimization. It cannot replace the products of the remaining plaquettes, and it cannot show flatness on its own.

2.4 Tests

The tests in `code/tests/test_holonomy_checker.py` check the following on S_3 and Q_8:

- the trivial connection is `FLAT_TRIVIAL` on open and periodic lattices;
- a connection that is a gauge transformation of the trivial one, with no plaquette of four identity links, is `FLAT_TRIVIAL`;
- a torus connection with a constant non-identity product around one cycle is `FLAT_NONTRIVIAL`;
- a single non-identity link away from the boundary gives `NOT_FLAT` with exactly two non-flat plaquettes;
- for every pair $(a, b) \in G \times G$, the 3×4 torus connection whose links from the last column to the first carry a, whose links from the last row to the first carry b, and whose other links are e, is flat exactly when $ab = ba$; when it is flat its cycle products are (a, b), or their conjugates by one element after a gauge transformation, and when it is not flat its only non-flat plaquette has product $bab^{-1}a^{-1}$;
- on seeded random connections, reversing a plaquette inverts its product, changing the starting vertex conjugates it, and some products differ from their inverses;
- skipping plaquettes with four identity links never changes the result;
- malformed group tables, lattices and link lists are refused, and the certificate check accepts a correct plaquette in either orientation and rejects a wrong product, a flat plaquette, a four-cycle that is not a plaquette and a contractible loop.

The seam family is small, but it is exhaustive over $G \times G$, and in it a flat connection with non-commuting cycle products never occurred, as the theory requires. The family contains 100 connections: 18 commuting S3 pairs and 40 commuting Q8 pairs. It tests 58 flat and 42 non-flat cases, not all connections on a lattice.

3. The C017 pilot

3.1 What was run

Challenge C017 asks for `NOT_FLAT`, `PURE_GAUGE` or `FLAT_NONTRIVIAL` verdicts with certificates for connections with values in S_3, Q_8 and $\mathrm{SL}(2, \mathbb{F}_3)$ on square lattices. Its specification calls for open-weight models of at most 70B parameters and 36 hidden base cases. The retained runs are smaller pilots. They used five cases from the challenge's judge packet (Table 1), one reply per case, and one model outside that specification. The five case files are copied into `code/data/c017/` with their SHA-256 digests and source paths.

Each arm sent one prompt per case and parsed a JSON object from the reply. Two checkers in the harness scored the certificate, and a case passed only when both accepted it.

- **Arm 1.** The lattice, the group table and inverses, all directed links, the four verdict definitions and the JSON format. Model `meta-llama/llama-3.3-70b-instruct`; temperature 0.1; at most 2,000 completion tokens.
- **Arm 2.** Arm 1 plus four written constraints: multiply in traversal order; test the product, not a sign or determinant; on a torus a flat connection need not be gauge-trivial; refuse inverse mismatches. Same model and settings.
- **Arm 5.** The Arm 2 constraints plus the full list of plaquettes in the vertex order that the checker accepts, an instruction to multiply step by step by table lookup, the two cycle paths on a torus and a decision procedure. Run with `meta-llama/llama-3.3-70b-instruct` (temperature 0.05, at most 3,500 completion tokens) and with `stealth/space-bunny-alpha` (temperature 0.05; the runner's default limit is 6,000 completion tokens, can be changed on the command line, and is not recorded in the report).
- **Arm 6.** Arm 5, except that the harness listed only the plaquettes that contain a non-identity link and stated that the others were flat. Run with `stealth/space-bunny-alpha` (temperature 0.05; runner default limit 4,000 completion tokens).

No C017 runs are retained for Arms 3 and 4. The model `stealth/space-bunny-alpha` is a new free "stealth" model offered on OpenRouter as of 2026-09-28; the provider does not disclose its identity, size or training. Results with it are exploratory. They fall outside the challenge protocol, SG-CP v1, whose §2 admits only open-weight models of at most 70B parameters (`omega_engine_projects/mssn_sn_symgam/challenges/01_challenge_protocol.md`), and they may not be reproducible once the stealth period ends. The reports record costs computed by the harness from configured per-token prices, not provider bills, so we do not report costs.

Table 1. The five retained C017 cases and the host verdicts computed by the package. "Skippable" counts plaquettes whose four links are the identity. The case files name the elements of S_3 as `e`, `r`, `r2`, `s`, `sr`, `sr2`, where `r2` is $r^2 = r^{-1}$, and the elements of Q_8 as `e`, `-e`, `i`, `-i`, `j`, `-j`, `k`, `-k`. The case `pois_01` comes from the challenge's poisoned set: its non-flat product is a 3-cycle, which has sign $+1$, so a test of the sign alone would call the connection flat.

Case	Group and lattice	Non-identity links	Plaquettes (skippable)	Host verdict
`pois_01`	S_3, 3×3 open	1 of 12	4 (3)	`NOT_FLAT`, $P_{0,0}$ has product `r2`
`base_01`	S_3, 3×4 open	1 of 17	6 (5)	`NOT_FLAT`, $P_{0,1}$ has product `s`
`base_02`	S_3, 4×5 open	28 of 31	12 (0)	`FLAT_TRIVIAL`
`base_03`	S_3, 5×6 torus	11 of 60	30 (20)	`FLAT_NONTRIVIAL`, $A =$ `r`, $B =$ `r2`
`base_13`	Q_8, 4×5 open	1 of 31	12 (11)	`NOT_FLAT`, $P_{0,0}$ has product `-i`

The host verdicts agree with the verdicts that the harness expected for all five cases. The S_3 table in the case files is isomorphic to the permutation group (the tests find all six isomorphisms), and the Q_8 table equals the package's quaternion table.

3.2 Outcomes

Table 2 gives, for each arm and case, what the model stated and whether the harness accepted the certificate. "70B" denotes `meta-llama/llama-3.3-70b-instruct` and "stealth" denotes `stealth/space-bunny-alpha`. "Reversed order" means that the answer named the correct non-flat plaquette with its vertices in the opposite orientation.

Table 2. Harness outcomes for the five C017 cases (one reply per cell).

Case	Arm 1, 70B	Arm 2, 70B	Arm 5, 70B	Arm 5, stealth	Arm 6, stealth
`pois_01`	reversed order	reversed order	accepted	accepted	accepted
`base_01`	reversed order	stated `PURE_GAUGE`	accepted	accepted	accepted
`base_02`	no parsable JSON	stated `NOT_FLAT`	flat plaquette stated as `r2`	flat plaquette stated as `r`	flat plaquette stated as `r`
`base_03`	loop product misstated	loop product misstated	no parsable JSON	reply cut off	accepted
`base_13`	reversed order	no parsable JSON	accepted	accepted	accepted
Accepted	0 of 5	0 of 5	3 of 5	3 of 5	4 of 5
Verdict label equal to host	4 of 5	2 of 5	3 of 5	3 of 5	4 of 5

For the 15 solution records retained in Arms 5 and 6, the package agrees with the historical harness: 10 accepted and 5 rejected. These records contain 13 parsed certificate objects and two `PARSE_ERROR` wrappers for replies whose JSON could not be parsed; rejecting a wrapper is not verification of an absent mathematical certificate. For Arms 1 and 2 the reports retain only the stated verdict and the checkers' reasons, not the certificates.

Vertex order. The Arm 1 and Arm 2 prompts defined a plaquette product as $U_{01}U_{12}U_{23}U_{30}$ but did not say in which order the checker lists a plaquette's vertices. The checker accepted only $(r,c),(r+1,c),(r+1,c+1),(r,c+1)$. Three Arm 1 answers named the correct non-flat plaquette in the order $(r,c),(r,c+1),(r+1,c+1),(r+1,c)$ and were rejected on that ground before any product was compared. The reports do not retain the stated products, so we cannot say whether they were right for the reversed order. Arm 1 therefore scored 0 of 5, although its verdict label matched the host in 4 of 5 cases. Two Arm 2 answers were rejected on the same ground; one of them also had the wrong verdict. The Arm 5 prompt listed the accepted order, and the three open non-flat cases were accepted for both models. Arm 5 also added the step-by-step instruction and the cycle paths, so these runs do not separate the effect of stating the convention from the other additions. The certificate check of Section 2 accepts either orientation.

The gauge-trivial case. `base_02` is flat and gauge-trivial, and 28 of its 31 links are not the identity. All three Arm 5 and Arm 6 answers for it stated `NOT_FLAT` with a plaquette whose product is e, stating the product as `r2` or `r`; each was rejected because the stated product disagrees with the table. The initial internal draft also stated that an instruction to build a gauge along a spanning tree biased models towards `PURE_GAUGE`. No retained run supports this, and the statement is withdrawn. The only retained `PURE_GAUGE` answer is the Arm 2 answer for `base_01`, a non-flat case, whose stated gauge fails on the edge $(0,1)\to(0,2)$.

The torus case. In `base_03`, 20 of the 30 plaquettes have four identity links, and each of the other 10 contains at least one of the 11 non-identity links. With Arm 5, the reply of `stealth/space-bunny-alpha` stopped at 8,000 completion tokens with its JSON unfinished; the retained start of the reply already carried the label `FLAT_NONTRIVIAL`. With Arm 6, where the harness listed only the 10 plaquettes with a non-identity link, the same model returned all 10 products (each e), $A =$ `r`, $B =$ `r2` and the label `FLAT_NONTRIVIAL` in 2,810 completion tokens, and both checkers accepted the answer. This is one case with one reply per arm. It shows that the shorter prompt fitted within the limit on this case; it does not measure a general saving. The Arm 6 listing relies on identity links, which a gauge transformation can remove (Section 2.3). The initial internal draft stated that 24 plaquettes were skipped and that the model evaluated only the two cycles. Recomputing the Arm 6 plaquette list from the case file gives 20 skipped plaquettes, and the retained reply evaluates the other 10 plaquettes and the two cycles.

The harness checkers. The two checkers in the C017 harness are not independent implementations. Both import the plaquette list and the path product from the challenge's reference solver. The check in this package shares no code with them, and on the retained solution records it reaches the same decisions. Agreement on these cases does not certify independence from shared mathematical assumptions or establish the soundness of either implementation beyond the checked scope.

3.3 Data sources for Section 3

All paths are relative to the repository root.

- Challenge specification: `omega_engine_projects/mssn_sn_symgam/challenges/b_cohomology/c017_nonabelian_lattice_holonomy.md`.
- Arms 1 and 2: `.../c017_nonabelian_lattice_holonomy/results/immune_transfer_report.json`, added in commit `7e639a0be573f6affc4d84a2aacb5573da92bc79`; runner `eval_epistemic_immune_transfer.py` at that commit.
- Arm 5 with `meta-llama/llama-3.3-70b-instruct`: `results/arm5_hybrid_report.json` as committed in `9e45ee501e6a48086aaabd2381632eef6f7ed7a2`. The file at that path was later

overwritten by a one-case run, so this report is retained only in the repository history.

- Arm 5 with `stealth/space-bunny-alpha`: `results/arm5_stealth_space_bunny_alpha_report.json`, commit `5327cfcb7ee3622e79f7d0cd884dfa37ef4eef56`.
- Arm 6: `results/arm6_stealth_space_bunny_alpha_report.json`, commit `eb5fc971ae44320ff8183f09343d547f830d9d56`.

The historical report digests are recorded with their original receipts. The byte copies in `docs/historical_evidence/` have pinned source commits, Git blob SHAs and SHA-256 digests in their provenance file. The package file `code/data/c017/model_claims.json` holds the verdicts, certificates, checker reasons and completion lengths extracted from these reports. `code/recount_pilots.py` freshly compares all 25 extraction entries with those source reports. That comparison does not recover the ten missing certificate payloads or generate new responses.

4. Six formal-mathematics pilots

4.1 What was run

For each of challenges C045 to C050, a runner script defines a ten-case panel, separate from the challenge's 64 hidden cases (for C049 panel B the script is not retained; see below), and sends each case once to three models through OpenRouter at temperature 0: `stealth/space-bunny-alpha` (the undisclosed stealth model described in Section 3.1), `meta-llama/llama-3.3-70b-instruct` and `meta-llama/llama-3.1-8b-instruct`. The reports do not record which provider served each model. A case passed only when both of the challenge's checkers, an exact checker and a second checker, accepted the answer.

The harness set the completion limits. In C045 and C046 the limit was 4,096 tokens for `stealth/space-bunny-alpha` and 2,048 for the other two models. In C047 and C049 panel A it was 4,096 tokens and in C048 and C050 2,048 tokens for all three. The limit for C049 panel B is not recorded.

Table 3. Cases passed, out of 10 per cell, one reply per case. SBA, 70B and 8B refer to the three model identifiers above, in that order.

Challenge	Task	SBA	70B	8B
C045	knot invariants from planar diagrams	2	8	3
C046	Knuth–Bendix completion and normal forms [4]	4	7	8
C047	linear recurrences or certified absence	4	2	0
C048	rational point or local obstruction for a ternary quadratic form	10	8	5
C049, panel A	limits, colimits and adjunctions in finite categories	5	6	4
C049, panel B	the same topics, a different set of ten cases	9	6	6
C050, run 1	equational rewriting proof or two-element countermodel	7	4	1
C050, run 2	the same ten cases, run again	6	3	1

The offline recount uses the per-case recorded flags. One C045 8B entry records `JSON_ERROR` without a pass flag; it is counted as an unaccepted response. This recount does not re-execute the historical certificate checkers.

C049 has two retained runs on different ten-case panels with the same three models, committed together on 2026-09-28. Panel A was produced by the runner script retained in the repository; the script that produced panel B is not retained. For `stealth/space-bunny-alpha` the two panels give 5 and 9 passes. C050 was run twice with the same committed runner and the same ten cases; the recorded start times are 98 seconds apart. The second report overwrote the first, which is retained in the repository history. For `stealth/space-bunny-alpha`, five of the ten cases changed outcome between the two runs, two of them because one run received an API error instead of a reply. The initial internal draft reported only C049 panel B and C050 run 2.

What the prompts disclosed. Each runner sent a case as the line `Case ID: <id>` followed by the case data (lines 219, 270, 221, 163, 264 and 236 of the C045 to C050 runners, unchanged from the commits that recorded the runs to the code baseline). The challenge authors wrote the identifiers, and many name the expected result. `c045_m02_trefoil_identify` asks for the identification of a knot that its identifier names. `c045_m09_refusal_corrupt_edge` and `c046_m08_refusal_commutativity` name a refusal and its reason. `c046_m05_check_non_confluent_group` names the confluence verdict. `c047_m07_prand_decoy` and `c047_m08_order9_decoy` mark the cases with no recurrence within the bounds. `c048_m01_solv_11_m2` and `c048_m02_solv_pyth` mark solvable forms, and `c048_m05_obstr_inf_111` names both the obstruction and its place. `c049_m05_no_product_disc2` names the verdict that no product exists. The C045, C046, C049 and C050 runners also sent the whole case record, including the `note` of the poisoned cases C049 m10 and C050 m09, whose identifiers call the note bait or fake; the C047 and C048 runners sent only the terms or coefficients and the bounds. Table 4 counts the identifiers of the six panels whose runner is retained. Which identifiers name the result is one reader's classification, the author's; it is published identifier by identifier with the replay listed in Section 4.4, so that another reader can check or dispute each case.

Table 4. What the case identifiers name, in the author's classification, and the most passes that an answer needing no mathematics obtains under the same rule (both checkers accept), out of 10 per panel. The answers tried were an empty object, a bare abstention, a refusal or out-of-scope answer with each reason the prompt lists, and a constant verdict where the output format allows one (Section 4.2).

Challenge	Names the verdict or answer	Names the object	Neither	Best baseline passes
C045	7	3	0	10
C046	5	3	2	10
C047	4	6	0	0
C048	10	0	0	3
C049, panel A	5	5	0	2
C050	5	5	0	1

In C045 and C046 the best such answer is a refusal of every case: both checkers accept a refusal with any of the listed reasons on every case, so it passes 10 of 10, more than any model in Table 3. In C048 a constant local obstruction at the place 2 passes 3 cases, and in C049 a constant "does not exist" passes 2. The identifiers of the C049 panel B report have the same form (for example `c049_m07_no_product_discrete`), but its runner is not retained, so what its prompts contained cannot be checked and its cases cannot be replayed.

The counts in Table 3 are therefore counts under leaked labels. These counts describe responses under the disclosed prompts and checker rules; they do not estimate or bound unaided performance. In C045 and C046 they do not separate any model from an answer that refuses every case. The models' own passes there include refusals only on the cases whose identifiers name a refusal (C045 m09 and m10; C046 m08 to m10), and the checkers accepted those refusals without comparing the stated reason with the input.

We do not add counts across challenges and we make no comparison between models. The tasks differ, the panels were written by the challenge authors, each cell has n = 10 and one reply per case, and no statistical test was run. The initial internal draft's totals over 80 cases and its conclusion that absence proofs remain a capability of symbolic algorithms alone are withdrawn; the evidence is 10 cases per cell on one day.

4.2 Observations

Harness limits. In C045 all 8 failures of `stealth/space-bunny-alpha` returned no answer object, with recorded token counts (prompt and completion together) of 4,751 to 4,845, consistent with the reply reaching the 4,096-token completion limit. In C046, 5 of its 6 failures were of the same kind, at 4,624 to 4,747 tokens. The reports do not record prompt and completion counts separately. These cases most likely show replies that did not fit the harness limit; they do not measure what the model could compute.

The second checkers. For those empty answers in C045 and C046 the second checker reported acceptance and only the exact checker rejected them. In C047 case m06, whose terms after the first are the Fibonacci numbers, `stealth/space-bunny-alpha` returned a certificate that no recurrence exists within the bounds; the second checker accepted it, and the exact checker rejected it because its minor used rows below the smallest index that the exact checker allows.

A replay of answers that need no mathematics, against the checkers as they were when the runs were recorded, shows how far this goes. It imports each challenge's two checkers from those commits, reads the panel from the runner without running it, and calls no model. The second checkers of C045 and C046 accept the empty object `{}`, a bare abstention, and any refusal that carries a reason and a witness, on all ten cases. The second checker of C047 accepts a recurrence with an empty coefficient list, and an absence certificate with an empty list of minor rows, on 9 of the 10 cases, because the determinant of an empty matrix is 1; on the tenth, whose terms include a non-integer, it raises an error. The second checker of C048 accepts any out-of-scope answer on all ten cases, and that of C049 accepts a constant "does not exist" on 8 of 10. A pass required both checkers. For the cited empty or malformed answers rejected by the exact checker, permissive acceptance by the second checker did not turn a rejection into a pass. This does not prove that the second checker adds no constraint on every possible answer. Table 3 remains a historical record of the original paired rule. In C045 and C046 even that rule accepts the constant refusals of Table 4.

The repository documents later harness corrections in commits `e6fbc15cf011121597a82a1d1996 26063bedf7c0` and `e2f7a3500413af960c6001832a1ab096a7a14af8`. The following is a historical summary of those changes, not a fresh checker execution in this review. The C045 and C046 second checkers now recompute the answer independently, so a refusal of every case, `{}` and an answer without content fail them; the C047 second checker no longer accepts an empty coefficient list or an empty list of minor rows; the C048 second checker accepts an out-of-scope answer only when a coefficient is not an integer; and the C043 to C050 runners replace the descriptive identifiers with opaque tokens drawn for each run. The C045 and C046 exact checkers still accept a refusal with any reason, so the pair now depends on the second checker for that case. None of this changes Table 3, which was produced with the checkers of the run commits.

Absence certificates. C047 certifies that no recurrence exists within the bounds by a nonsingular minor, whose determinant the exact checker computes by fraction-free elimination [5]. On the three panel cases that the runner marks as having no recurrence within the bounds (m07, m08, m10), no model returned a certificate that both checkers accepted. `stealth/space-bunny-alpha` returned no answer content for all three; the other two models stated recurrences or malformed certificates.

Other failures. In each C050 run, one failure of `stealth/space-bunny-alpha` was an API error with no reply. In C048, the two failures of `llama-3.3-70b-instruct` were integer points that do not satisfy the form's equation.

4.3 What the checkers and mutation suites show

Each challenge's judge packet retains 64 hidden cases and 48 mutated certificates for testing the checkers, 512 and 384 in total over C043 to C050. Scoring receipts are retained for C043 and C044 only. Each records the reference solver passing all 64 hidden cases, the two checkers agreeing on all of them, and all 48 mutations rejected. No scoring receipt is retained for C045 to C050, so the initial internal draft's statement that 512 cases were verified and 384 mutations rejected is narrowed to these 128 cases and 96 mutations. The receipts concern the reference solvers and checkers, not the models.

Challenges C043 and C044 were also run with the same three models. The C043 runner wrote its report to a directory that the repository's ignore rules exclude. The C044 runner wrote its report outside the repository, to a directory it reaches six levels above its own file. Neither report is under version control, and the counts appear only in narrative findings documents, so we do not report them. They would not measure identification in any case: the C044 runner sent each example file

whole, including its `note`, which names the group or the expected verdict (for example "Quaternion group Q8", "C4 against Klein four group V4" and "S5 has order 120 which exceeds max allowed order 64"). The C043 prompts were built from the case fields and carried no note. An earlier internal review reported an external file with the C044 report's name (SHA-256 `5fecd63f08dc1b5b2094fe6b722c236f284567b52a9a140f62d4825e63e8648b`); it was not under version control when the run was recorded. Its presence and bytes were not independently checked in this review, and we do not report its counts. In the corrected harness the C044 runner no longer sends the `note` or reads a credentials file, and it writes its report into the repository's run store.

4.4 Data sources for Section 4

All paths are relative to `omega_engine_projects/mssn_sn_symgam/challenges/e_formal_math/` .

- C045 run report: `c045_knot_polynomials_from_pd/empirical_results/c045_space_bunny_matrix_report.json`, commit `ea2b55d841bdbe283f2f6ac77abe8a1677cbf0f8`.
- C046, C047 and C048 run reports: the files with the same pattern in their challenge folders, commit `46bd1dec156260f073b77caa07d8a453f8efad30`.
- C049 panel B: `c049_finite_category_universal_constructions/empirical_results/c049_space_bunny_matrix_report.json`; panel A: the three files `results_*.json` in the same folder; both in commit `46bd1dec156260f073b77caa07d8a453f8efad30`.
- C050 run 1: `c050_equational_theorem_proving/empirical_results/c050_space_bunny_matrix_report.json` as committed in `46bd1dec156260f073b77caa07d8a453f8efad30`; run 2: the same path as committed in `5a256c6e8d088c612332362a8abc53138806ac7a`.
- Runner scripts, which set the panels, the prompts and the completion limits: `run_c045_matrix.py` to `run_c050_matrix.py` in the challenge folders.
- The replay behind Table 4 and Section 4.2 (paths relative to the repository root): the collection's script `omega_engine_atlas/coda_atlas/omega_engine_symbol_game/tools/replay_formal_math_baselines.py` and its record `omega_engine_atlas/coda_atlas/omega_engine_symbol_game/verification/formal_math_baselines.json`. It reads the runners and checkers from commits `ea2b55d841bdbe283f2f6ac77abe8a1677cbf0f8` (the C045 runner) and `46bd1dec156260f073b77caa07d8a453f8efad30` (C046 to C050). The C045 checkers are read from commit `c568c20062697b54ddb8729e2476d4cfdb5c4982`, which differs from the run commit in that folder only by one import line that Python 3.12 needs to load the exact checker.
- Scoring receipts: `c043_ramsey_vdw_constructions/commit_reveal/c043_l41_scoring_receipt.json` (commit `50197478726e5ecb791e519760cee9a79af2af27`) and `c044_finite_group_identification/commit_reveal/c044_l41_scoring_receipt.json` (commit `999b36c1afed0f905f634f83adc1c646da7b975e`).

5. Limitations

The C017 pilot has five cases, one reply per case and arm, two models, and arms that change several things at once. The formal-mathematics pilots have ten cases per cell and one reply per case, and their prompts named the expected result for 36 of the 60 cases whose prompts can be reconstructed (Table 4). Neither pilot supports a claim that one prompt or model is better than another, or any claim about unaided capability, and none is made. Replies from hosted models can vary between calls even at temperature 0.

One of the three models is undisclosed and may not remain available. Two reports (C017 Arm 5 with `meta-llama/llama-3.3-70b-instruct` and C050 run 1) were overwritten in the repository history; byte copies are now included in this package. The C043 and C044 model reports and the C049 panel B runner remain unavailable in the version-controlled evidence reviewed here. The repeated C050 panel shows that a single reply per case can change between runs; C049 instead compares different panels.

The package freshly replays the C017 cases and 15 retained solution records. Its offline recount verifies source custody, extraction and table arithmetic: Table 3 retains original acceptance flags, while Table 4 retains author annotations and the prior checker replay record. Fresh C045–C050 checker execution still requires a repository checkout with the cited commits. The C017 harness checkers share reference-solver code. Lean formalizes only the Section 2.3 remark; the complete host is tested, not mechanically formalized. This review did not execute the pinned Lean compiler; historical Lean reports are retained without being presented as fresh evidence.

This paper assigns no authorization and changes no jurisdiction. It carries no operational authority or institutional standing.

6. Reproduction

From the package root with Python 3.12 or later, install the development tools and run the offline checks (on Windows, substitute `py -3.12` for `python`):

```
python -m pip install -e '.[dev]'
python -m pytest code/tests
python -m ruff check code tools/study_capstone.py tools/render_paper.py
python -m mypy code
python code/replay_c017.py
python code/recount_pilots.py
python tools/study_capstone.py --output /tmp/paper010-capstone.json
python tools/render_paper.py
python verify_deposit.py
python tools/rebuild_release.py --check
```

Choose a writable capstone output path on Windows. The development extra lists pytest 9 or later, ruff 0.14.13 and mypy 1.19.1; the current receipt records the versions actually used. Runtime algebra and the replay/recount scripts use only the standard library and packaged data. They make no provider or network call.

The deposited renderer uses Pandoc and XeLaTeX to regenerate both `paper.tex` and `paper.pdf`. To rebuild release custody after an intentional edit, run `python tools/rebuild_release.py`, then both deposit checks again. Digest agreement establishes file custody, not the truth of all manuscript claims.

With the exact toolchain in `lean4/lean-toolchain` (Lean 4.34.0-rc1), run `lake build` in `lean4/`, then `python tools/replay_lean_axiom_census.py --replay .` from the package root. A static `--check .` only compares the generated census source with the retained named report. The separate full-module report includes generated structure declarations; it is not a second mathematical theorem or a proof of the complete checker. The historical C045–C050 checker replay needs the collection folder and cited Git commits: `python tools/replay_formal_math_baselines.py --check`. That operation is distinct from this package's offline report recount.

Verification for this revision

`docs/v01_review_2026_10_02.json` records the reviewed repository baseline, commands, actual tool versions, exit statuses and scope. Current tests, lint, type checking, C017 replay, source/extraction/table recount, capstone, PDF rendering and deposit custody checks were executed. The pinned Lean build and fresh C045–C050 checker execution were not run in this environment, and no model trial was made. The earlier `docs/release_validation_*.json` files and Lean reports retain their original dates and scope; they are historical records. The current review record makes these limits explicit rather than treating a retained report as execution.

References

1. K. G. Wilson, "Confinement of quarks," *Physical Review D* **10**(8), 2445–2459 (1974). doi:10.1103/PhysRevD.10.2445
2. J. B. Kogut, "An introduction to lattice gauge theory and spin systems," *Reviews of Modern Physics* **51**(4), 659–713 (1979). doi:10.1103/RevModPhys.51.659
3. A. Hatcher, *Algebraic Topology*. Cambridge University Press, 2002, Chapter 1. Author's book page
4. D. E. Knuth and P. B. Bendix, "Simple word problems in universal algebras," in J. Leech (ed.), *Computational Problems in Abstract Algebra*, pp. 263–297. Pergamon Press, 1970. doi:10.1016/B978-0-08-012975-4.50028-X
5. E. H. Bareiss, "Sylvester's identity and multistep integer-preserving Gaussian elimination," *Mathematics of Computation* **22**(103), 565–578 (1968). doi:10.1090/S0025-5718-1968-0226829-0